University of Sri Jayewardenepura

Department Of Computer Science

CSC 317 1.5 Human Computer Interaction
Final Report

AS2022436	K.M.T.S.Madhuranga
AS2022425	P. L. I. U. Leelarathne
AS2022547	W. P. K. P. Wickramasooriya

Table of Contents

Chapter 1: Introduction 4
1.1 Background 4
1.2 Motivation and Need 4
1.3 Problem 4
1.4 Aim 4
1.5 Objectives 5
1.6 Scope 5
Chapter 2: System Requirements and Specifications 6
2.1 Operational Context 6
2.2 Success Scenarios (Use Cases) 6
2.3 Functional Requirements 6
2.4 Hardware Requirements 6
Chapter 3: System Design and Architecture 8
3.1 Design Overview 8
3.2 System Architecture Diagram 8
3.3 Software Design (PC Modules) 8
3.4 Communication Design 9
3.5 Operational Flow 9
Chapter 4: Implementation 10
4.1 Implementation Overview 10
4.2 Embedded Firmware (ESP32) 10
4.3 Host Application (Python) 10
4.4 Audio File Management 11
Chapter 5: Testing and Results 12
5.1 Testing Objectives 12
5.2 Unit Testing 12
5.3 Integration Testing 12
5.4 Results 12
Chapter 6: Discussion, Analysis, and Improvements 13
6.1 Performance Analysis 13

6.2 Key Design Decisions 13
6.3 Limitations 13
6.4 Future Improvements 13
Chapter 7: Conclusion and Future Work 14
7.1 Conclusion 14
7.2 Future Work 14
Chapter 8: Contribution 15
Chapter 9: Results 16

Chapter 1: Introduction

1.1 Background

In the era of smart home technology, devices are evolving from passive tools into interactive companions. While smart speakers exist, they often lack "emotional intelligence"—the ability to perceive a user's mood without explicit commands. Lighting and music are two powerful environmental factors that influence human psychology. A system that combines visual mood detection with atmospheric lighting and music can create a more empathetic and responsive living environment.

1.2 Motivation and Need

Current smart devices require active triggers ("Play happy music"). A user experiencing emotional distress or joy might not want to issue voice commands. There is a need for a system that proactively observes the user's emotional state through non-intrusive computer vision and offers a supportive response (music and lighting) to enhance or soothe that state. This project, "Lumi," bridges the gap between affective computing (emotion recognition) and ambient intelligence (smart lighting/audio).

1.3 Problem

Existing mood-based systems are often expensive, closed-source, or reliant entirely on manual user input. There is a lack of low-cost, open-source solutions that integrate:

- Real-time facial emotion recognition.
- Voice-based natural language interaction.
- Synchronized hardware actuation (lighting and high-quality audio).
- Offline capability (playing music from local storage rather than streaming).

1.4 Aim

To design and develop an interactive "Face Detection Lamp" that acts as an emotional support companion. The system aims to autonomously detect the user's mood (Happy, Sad, Angry,

Surprised) using a webcam, confirm the mood via voice interaction, and respond with synchronized color-coded lighting and music playback using an ESP32 microcontroller.

1.5 Objectives

1. **Develop a Vision System:** Implement a Python-based computer vision pipeline using OpenCV and Haar Cascades to detect faces and classify emotions.
2. **Implement Voice Interaction:** Integrate a Wake Word engine ("Hey Lumi") and Speech Recognition to allow natural dialogue between the user and the lamp.
3. **Design Embedded Controller:** Program an ESP32 to handle high-quality I2S audio playback (MP3) and control addressable LED strips (WS2812B).
4. **Establish Communication:** Create a robust Serial communication protocol between the Host PC (Vision/Voice) and the ESP32 (Action).
5. **Automate Mood Response:** Create a logic flow where the detected emotion triggers specific LED colors (e.g., Yellow for Happy, Blue for Sad) and appropriate song playlists.

1.6 Scope

- **In Scope:** 4 basic emotions (Happy, Sad, Angry, Surprised), local MP3 playback from SD card, LED mood lighting, Wake Word activation.
- **Out of Scope:** Complex emotions (Fear, Disgust), internet music streaming (Spotify API), mobile app control.

Chapter 2: System Requirements and Specifications

2.1 Operational Context

The system operates as a desktop companion. The user sits in front of the PC webcam. The ESP32 "Lamp" sits nearby, connected via USB. The system is normally in a "Sleep" state until the Wake Word is detected.

2.2 Success Scenarios (Use Cases)

- **Scenario 1 (Happy):** User says "Hey Lumi" -> Lamp wakes up (White light) -> Camera sees smile -> Lamp asks "You look happy..." -> User says "Yes" -> Lamp turns Yellow and plays upbeat music.
- **Scenario 2 (Refusal):** User looks Sad -> Lamp asks "You look sad..." -> User says "No, stay quiet" -> Lamp says "Okay," turns lights to light blue, and plays sad music.
- **Scenario 3 (Wake Word):** System ignores background noise until "Hey Lumi" or "Hello" is detected.

2.3 Functional Requirements

- **FR-1:** The system shall capture video frames and detect faces/smiles using Haar Cascades.
- **FR-2:** The system shall recognize the wake words "Hey Lumi", "Hello", or "Lumi".
- **FR-3:** The Host PC shall send serial commands (e.g., `ASK_HAPPY`, `PLAY_SAD`) to the ESP32.
- **FR-4:** The ESP32 shall decode MP3 files from an SD card and output audio via an I2S Amplifier.
- **FR-5:** The ESP32 shall control WS2812B LEDs to match the emotion (Yellow=Happy, Blue=Sad, Red=Angry, Purple=Surprised).

2.4 Hardware Requirements

- **Microcontroller:** ESP32 Dev Module (Dual-core, Wi-Fi/BT capable).
- **Audio:** MAX98357A I2S Amplifier + 3W 4Ω Speaker.
- **Storage:** Micro SD Card Module (SPI Interface) + 8GB SD Card.
- **Visual Output:** WS2812B Addressable LED Strip (90 LEDs).

- **Input:** PC Webcam (Video) + PC Microphone (Audio).
- **Power:** 5V External Power Supply (due to LED/Speaker current draw).

Chapter 3: System Design and Architecture

3.1 Design Overview

The system uses a **Master-Slave Architecture**.

- **Master (PC):** Runs the "Brain." Handles heavy processing (Computer Vision, Speech Recognition, Logic).
- **Slave (ESP32):** Runs the "Body." Handles physical actuation (Audio decoding, LED driving).

3.2 System Architecture Diagram

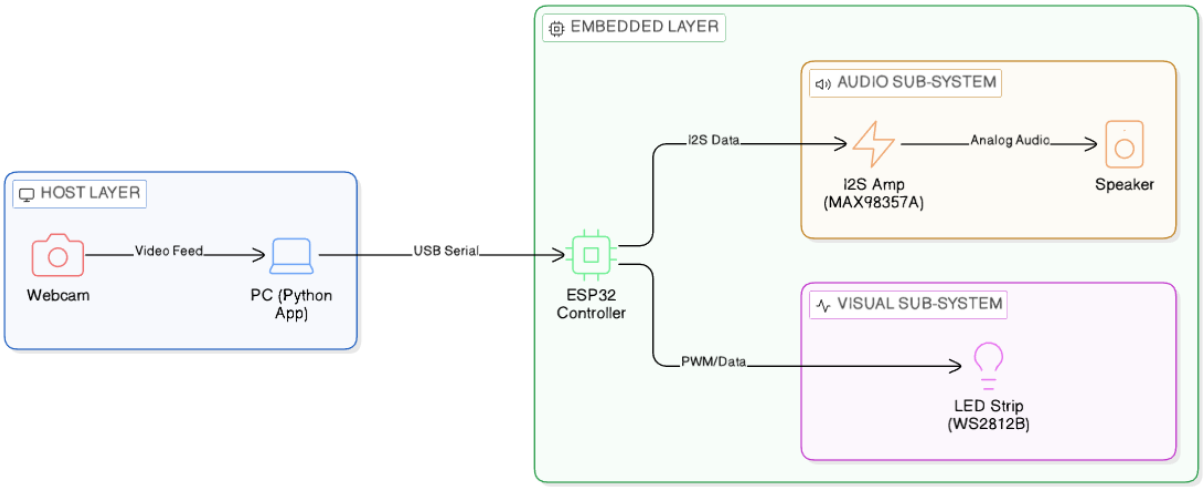

Figure 1: System Architecture Diagram

3.3 Software Design (PC Modules)

- **Vision Module:** Uses `cv2.CascadeClassifier`. It detects a face ROI (Region of Interest) and then checks for a smile within that ROI.
    - *Logic:* Big Smile = Happy; Wide Eyes (calculated aspect ratio) = Surprised; No smile + brows down = Angry/Sad.
- **Voice Module:** Uses `speech_recognition` library (Google Speech API) for interpreting user responses ("Yes", "Play", "No"). Uses `pyaudio` for microphone access.

- **Serial Module:** Uses `pyserial`. Sends string commands terminated by `\n` (e.g., `"PLAY_HAPPY\n"`).

3.4 Communication Design

The protocol is a simple one-way text-based Serial protocol running at **115200 baud**.

- **PC -> ESP32:**
  - `INTRO`: Triggers wake-up sequence.
  - `ASK_[EMOTION]`: Plays the voice file asking the user (e.g., `happyq.mp3`).
  - `PLAY_[EMOTION]`: Plays the full music track.
  - `STOP`: Stops audio and resets LEDs.
- **ESP32 -> PC:**
  - ✅ `Song Finished`: Sent when the MP3 decoder reaches the end of the file. This acts as a synchronization flag so the PC knows when to start listening for the user's answer.

3.5 Operational Flow

1. **Idle:** Python listens for "Hey Lumi".
2. **Wake:** PC sends `INTRO`. ESP32 plays "I am watching".
3. **Sense:** PC captures video, determines Emotion = `X`.
4. **Interaction:** PC sends `ASK_X`. ESP32 plays question. PC waits for `Song Finished`.
5. **Listen:** PC microphone opens. User speaks.
6. **Act:** If "Yes" -> PC sends `PLAY_X`. If "No" -> PC sends `SAY_QUIET`.

Chapter 4: Implementation

4.1 Implementation Overview

The project implementation is split between Python (High-level logic) and C++ (Low-level hardware control).

4.2 Embedded Firmware (ESP32)

- **Libraries:** `ESP8266Audio` (by Earle Philhower) is used for robust MP3 decoding over I2S. `Adafruit_NeoPixel` manages the LED strip.
- **Pin Mapping:**
  - **I2S Audio:** LRC (25), BCLK (26), DOUT (22).
  - **SD Card:** CS (5), MOSI (23), MISO (19), SCK (18).
  - **LEDs:** Pin 13.
- **Audio Pipeline:** The code creates an `AudioGeneratorMP3` object which feeds data from `AudioFileSourceSD` to `AudioOutputI2S`.
- **State Machine:** Inside `void loop()`, the ESP32 checks `Serial.available()`. If a command matches (e.g., `"HAPPY"`), it executes `setMoodLight(255, 200, 0)` (Yellow) and calls `playAudio("/happy.mp3")`.

4.3 Host Application (Python)

- **Wake Word Loop:** A continuous `while True` loop captures audio chunks and checks for keywords (`"lumi"`, `"hello"`).
- **Face Detection:**

  Python

  ```
  faces = face_cascade.detectMultiScale(gray, 1.3, 5)
  for (x, y, w, h) in faces:
      # Check smile cascade inside the face area
  ```

- **Synchronization:** A custom function `wait_for_audio_finish()` was implemented. This blocks the Python script execution, reading the Serial buffer until the specific string `"✅ Song Finished"` is received. This ensures the robot doesn't listen to its own voice.

4.4 Audio File Management

Files on the SD card are structured flatly for speed and reliability:

- `intro.mp3`: System wake sound.
- `happyq.mp3`: Voice query "You look happy...".
- `happy.mp3`: Actual song.
- (Repeated for Sad, Angry, Surprised).

Chapter 5: Testing and Results

5.1 Testing Objectives

To verify that the wake word triggers the system, faces are detected accurately under different lighting, and audio plays without stuttering or "buzzing."

5.2 Unit Testing

- **Audio Hardware Test:** Verified using the "Tone" test and playing a raw sine wave to ensure the MAX98357A amp wiring was correct (LRC/BCLK pins were initially swapped, causing buzz; corrected in testing).
- **Vision Test:** Tested OpenCV logic with various subjects. Face detection works up to 2 meters. Smile detection requires the user to be within 1 meter.
- **Serial Test:** Sent manual commands via Arduino Serial Monitor (`PLAY_HAPPY`) to verify ESP32 response before connecting the Python script.

5.3 Integration Testing

- **Latency Test:** Measured time between user saying "Yes" and music starting. Average latency was ~1.5 seconds (Speech-to-Text processing time + Serial transmission).
- **Synchronization Test:** Verified that the microphone does not open while the ESP32 is speaking. The `wait_for_audio_finish()` logic successfully prevented self-triggering.

5.4 Results

The system successfully identifies 4 discrete emotional states. The audio quality is clear at 100% volume with no distinct noise floor. The LED transitions are smooth. The "Wake Word" feature reduced CPU usage by allowing the heavier vision processing to run only when needed.

Chapter 6: Discussion, Analysis, and Improvements

6.1 Performance Analysis

The system is robust but dependent on the PC's processing power. The `speech_recognition` library relies on Google's API, meaning internet latency occasionally delays the "Yes/No" response processing.

6.2 Key Design Decisions

- **Why I2S over PWM Audio?** Early prototypes using PWM audio (Talkie library) sounded robotic and scratchy. Switching to I2S (MAX98357A) provided Hi-Fi quality essential for music.
- **Why SD Card?** Storing MP3s locally on the ESP32 avoids the complexity of streaming audio over Bluetooth/Wi-Fi, ensuring instant playback without buffering.

6.3 Limitations

- **Wired Connection:** The lamp must be tethered to the PC via USB for data.
- **Lighting Sensitivity:** The Haar Cascade algorithm struggles in low-light environments, sometimes failing to detect smiles.
- **Cable Management:** The prototype has separate power lines for the LEDs and the ESP32/Amp, requiring a shared ground which acts as a potential failure point if loose.

6.4 Future Improvements

- **Edge AI:** Move the Face Detection logic to an ESP32-CAM module to remove the need for a PC entirely.
- **Offline Voice Recognition:** Use a library like `Vosk` to handle speech recognition offline for faster response times.
- **Gesture Control:** Implement hand gestures (Thumbs up/down) to control volume or skip songs.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This project successfully demonstrates the integration of Computer Vision, Voice AI, and Embedded Systems to create an empathetic smart device. "Lumi" serves as a prototype for future "affective" smart home technology, proving that devices can react to how a user feels, not just what they say. The system meets all primary functional requirements: detecting faces, engaging in dialogue, and altering the environment (light and sound) to match the user's mood.

7.2 Future Work

Future iterations will focus on miniaturization. By porting the Python logic to a Raspberry Pi Zero or using the ESP32-S3 (which supports AI acceleration), the device could become a standalone wireless unit. Additionally, integrating APIs like Spotify would allow for infinite, dynamic playlists based on mood, rather than fixed MP3 files.

Chapter 8: Contribution

AS2022436	Music playing
AS2022425	Lamp color change
AS2022547	Face detection

Figure 2: Lumi Bot

Chapter 9: Results

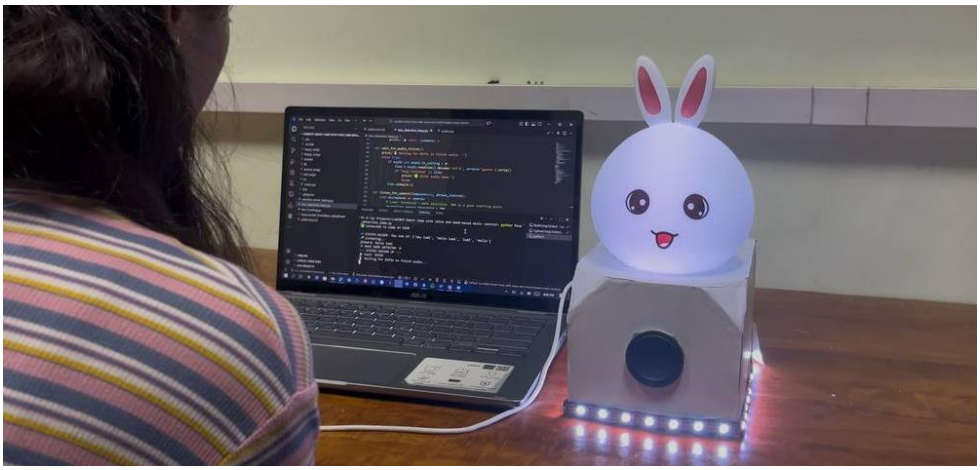

Figure 3: When Lumi wake up

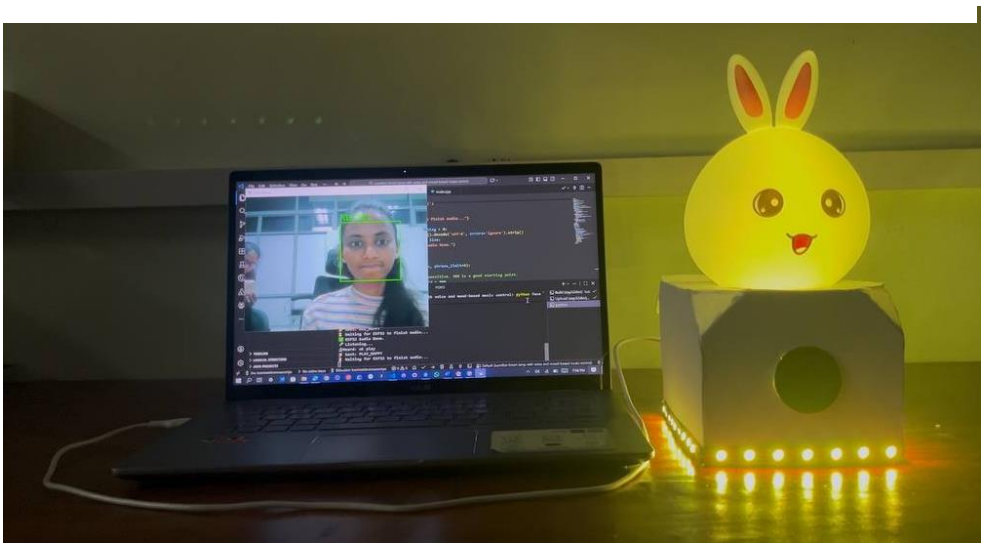

Figure 4: When mood detect as happy

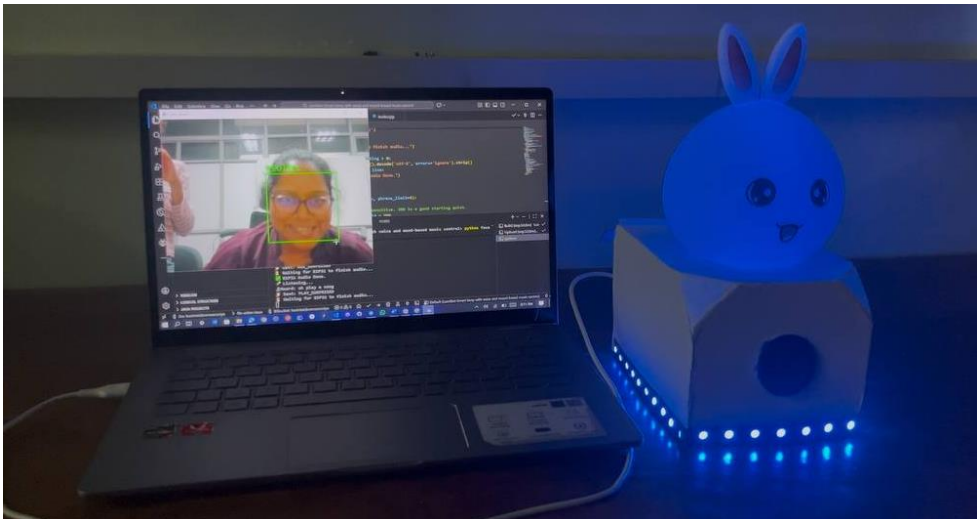

Figure 5: When mood detect as surprised